

Introduction to Python



Agenda

- ❑ **What is Python? Why Python? How to use it?**
- ❑ **Python basics**
 - ❑ Lists
 - ❑ Tuples
 - ❑ Dictionary and sets
 - ❑ Functions
 - ❑ Classes
 - ❑ Try-Catch exception

Agenda

❑ **What is Python? Why Python? How to use it?**

- ❑ Python basics
- ❑ Lists
- ❑ Tuples
- ❑ Dictionary and sets
- ❑ Functions
- ❑ Classes
- ❑ Try-Catch exception



What is Python?

Python is a High-Level and General Purpose **programming language**.

- High-level: Python is designed to be **easy for humans to read/write codes**.
- General Purpose: Python can be used for a **wide range of applications**, from web development and **data analysis** to **machine learning**.



Why Python?

Python is widely used in the field of Data Science and AI*.

Below are some of the main reasons why:

- ❑ **Readability** (Easy to read & write codes - similar syntax to English)
- ❑ Huge **Libraries and Frameworks** (For DS* and ML*)
- ❑ **Large and Active Community** (of Developers)
- ❑ **Compatibility** with Major Platforms

How to use it?

Firstly, you need to have a Python interpreter installed on your computer.

To download, follow <https://www.python.org/downloads/> which will provide a button to download an installer for your particular system.

The Python documentation also has a detailed guide on how to install and setup Python here: <https://docs.python.org/3/using/index.html>

Or you can use Google collab for easy approach:

<https://colab.research.google.com/>



How to use it?

If you install either Python interpreter or Anaconda on your computer, you may also want to install an **IDE (Integrated development environment)** software on your computer.

IDE makes it easier for you to **build, edit, or debug your Python code**.

One of the most popular IDE used is **Visual Studio Code (VSCode)**:

<https://code.visualstudio.com/download>

Alternatively, you can also try out **JupyterLab**:

<https://jupyter.org/install>



Visual Studio Code



What do you need to do? (Step by Step)



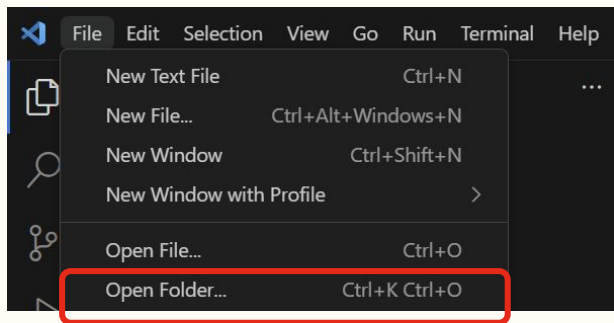
- 1) **Install Python** (<https://www.python.org/downloads/>)
- 2) **Install VSCode** (<https://code.visualstudio.com/download>)
- 3) **Open** Visual Studio Code

**15 - 20
Mins**

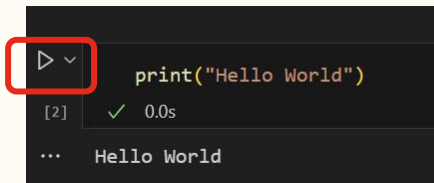
What do you need to do? (Step by Step)



1. Go to **File > Open Folder...** to select your coding workspace



2. Create a new file and name it **"Lab for Week 1.ipynb"** and open it.
3. Let's try printing "Hello World" Together. Type code and click **Run** like below.



Agenda

- ❑ What is Python? Why Python? How to use it?
- ❑ **Python basics**
- ❑ Lists
- ❑ Tuples
- ❑ Dictionary and sets
- ❑ Functions
- ❑ Classes
- ❑ Try-Catch exception

Assigning Variables

Assigning variables one by one

```
x = 3  
y = 4  
print(x+y)
```

Output:

7



Assigning Multiple variables in one

```
x, y = 3, 4  
print(x+y)
```

Output:

7



Best Practices

Best practices and tips for writing python codes

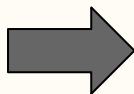
- For Variables, use **lowercase** with words separated by underscores.
 - `lives_left = 3`
- For Constants, use **all uppercase letters** with words separated by underscores.
 - `MAX_LIVES = 3`
- You can read more best practices at the official Style Guide for Python Code (<https://peps.python.org/pep-0008/>)
- Remember that the style guide is a "**Convention**" and **Not a strict "Rule"** enforced by the language.

Basic Data Types

In Python, there are four basic **data types**: **Integer**, **Float**, **String**, and **Boolean**

```
a = 3           #integer  
b = 4.5         #float  
c = "Hello"     #string  
d = True        #boolean
```

```
print(type(a))  
print(type(b))  
print(type(c))  
print(type(d))
```



Output:

```
<class 'int'>  
<class 'float'>  
<class 'str'>  
<class 'bool'>
```

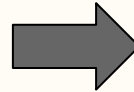


Note: In Python, anything following a *hash symbol (#)* on a line is a comment and is ignored by the interpreter.

Output Variables

Basic Ways:

```
w = "World!"  
  
print(w)  
  
#Using Plus Sign  
print("Hello" + w)  
print("Hello" + " " + w)  
  
#Using Comma  
print("Hello", w)
```



Output:

```
World!  
  
HelloWorld!  
Hello World!  
  
Hello World!
```

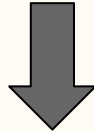


Output Variables

Print using F-strings:

In Python 3.6 and above, **F-strings** was introduced. They are easier to read and write than older methods because the variables are right where you want them to appear in the output string.

```
name = "Anushka"  
country = "Nepal"  
#Using an f-string to print  
greeting = f"Hello, {name}! You are from {country}."  
print(greeting)
```



Output:

Hello, Anushka! You are from Nepal.

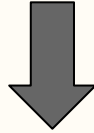


Output Variables

Print using F-strings:

F-strings can be used to control decimal place as well in numbers.

```
PI = 3.14159  
print(f"The value of Pi is: {PI:.2f}")
```



Output:

The value of Pi is: 3.14



Output Variables

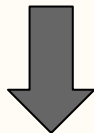
How to print with Quotations marks

#Without Quotation Mark

```
print("He said, Nice weather today")
```

#With Quotation Mark

```
print("He said, \"Nice weather today\"")
```



Output:

He said, Nice weather today

He said, "Nice weather today"



Boolean Data type

Booleans can represent only one of two values: **True** or **False**.

In programming you sometimes want to know if an expression is True or False.

```
5 > 3
```

Output: True

```
10 > 20
```

Output: False

```
5*3*3*21*47 > 5*2*42*36
```

Output: ?????



Logical Operators

and operator:

- checks if both conditions are True.
- If one condition is False, the entire expression is False.

True and True

Output: True

True and False

Output: False

False and True

Output: False

False and False

Output: False

or operator:

- checks if at least one of the conditions is True
- The entire expression is True as long as one of the conditions is True.

True or True

Output: True

True or False

Output: True

False or True

Output: ????



False or False

Output: ????



Comparison Operators

Python supports comparison conditions from mathematics.

Comparison Operators are the tools we use to compare values.

- Equals: $a == b$
- Not Equals: $a != b$
- Less than: $a < b$
- Less than or equal to: $a <= b$
- Greater than: $a > b$
- Greater than or equal to: $a >= b$

```
a = 10  
b = 5  
a == b
```

Output: False

```
a = 10  
b = 5  
a != b
```

Output: True

```
a = 10  
b = 5  
a < b
```

Output: False

If Statements

if statements are the "**decision-makers.**"

They take the answer from a logical operator and decide what to do next.

```
a = 10
b = 5
if a == b:
    print ("a is equal to b.")
```

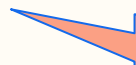
INDENT

Output:

```
a = 10
b = 5
if a != b:
    print ("a is not equal to b.")
```

INDENT

Output: a is not equal to b.



Python relies on indentation to define scope.

DO NOT Forget to add indentation after the if statement.

If Else Statements

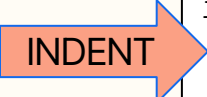
The if... else statement evaluates a condition.

If the condition is **true**, the code block directly **under the if statement is executed**.

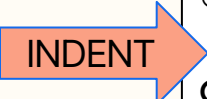
If the condition is **false**, the code block directly **under the else statement is executed**.

```
a = 10
b = 5
if a == b:
    print ("a is equal to b.")
else:
    print ("a is not equal to b.")
```

Output: a is not equal to b.



INDENT



INDENT



logical operator + If else

```
#Program to identify Class Cancellation
today    = "Tuesday"
holiday  = True

if today == "Tuesday" and holiday:
    print("Class is CANCELED! Time to relax!")
else:
    print("Class is ON. See you there!")
```

Output: Class is CANCELED! Time to relax!



```
#What will the output change to if holiday variable changes?
today    = "Tuesday"
holiday  = False
```

Output: ??????



=== Quick Quiz: Conditional Logic ===



24

Time: 3 Minutes

1. Is it a School Day?

The program should print "Come to ENG Class!" on Monday.

The program should print "Come to CP Class!" on Tuesday.

For the rest of the week, the program should print "Stay home".

```
today = "Tuesday" #change this value to all 7 days and see if it works
```

```
if [redacted]:
    print("Come to ENG Class!")
elif [redacted]:
    print("Come to CP Class!")
else:
    print("Stay home!")
```


=== Quick Quiz: Conditional Logic ===

SOLUTION

```
today = "Monday" #variable

if today == "Monday":
    print("Come to ENG Class!")
elif today == "Tuesday":
    print("Come to CP Class!")
else:
    print("Stay home!")
Output: "Come to ENG Class!"
```

Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

```
3 + 3 #6
3 - 3 #0
3 * 3 #9
3 / 3 #1.0 (Float)
```

```
3 % 3 #0
3 ** 3 #27
3 // 3 #1 (Integer)
```

Import Libraries

You may want to exactly use external libraries to solve some specific problems.
For example, a library called **math** provides access to **extra mathematical functions**.
<https://docs.python.org/3/library/math.html>

To use the functions in the library, you need to import the library first.
Example: you want to find square root of 25. How can you do it?

```
import math
x = math.sqrt(25)
print (x)
Output: 5.0
```



Another way:

```
import math as m #use 'as' to give the library a nickname
x = m.sqrt(25)
print (x)
Output: 5.0
```



User Input

We can also receive **user input as value** to be used in our program:

```
name = input("Please give me your name:")  
print("Your name is:", name)
```

Output: Your name is:Anushka



#What could be the problem?

```
num = input("Give me a random number:")  
print("I will add this number with 10")  
new_num = num + 10  
print(new_num)
```



User Input

#What could be the problem?

```
num = input("Give me a random number:")  
print("I will add this number with 10")  
new_num = num + 10  
print(new_num)
```



Output: -----

```
TypeError                                Traceback (most recent call last)  
Cell In[51], line 3  
      1 num = input("Give me a random number:")  
      2 print("I will add this number with 10")  
----> 3 new_num = num + 10  
      4 print(new_num)
```

TypeError: can only concatenate str (not "int") to str

Why the Error?

Because the input() method receives user input as a **string**.

string can not be added directly with an **integer** value.

To solve this problem, a **data type conversion** is needed.

Data Type Conversion

- `int(x)` converts **x** to integer
- `float(x)` converts **x** to float
- `str(x)` converts **x** to string
- `bool(x)` converts **x** to boolean

#Problem Resolved

```
num = input("Give me a random number:")  
print("I will add this number with 10")  
new_num = int(num) + 10  
print(new_num)
```



Output: #If I enter 5 as input
I will add this number with 10
15

Data Type Conversion

```
x = True #Boolean  
int_x = int(x)  
print(int_x)
```

Output: 1

```
x = False #Boolean  
int_x = int(x)  
print(int_x)
```



Output: ??

While Statement

while $\Rightarrow$ as long as the condition is met, keep doing it.

Use Case: Until correct password is given, ask user to enter password over and over again.

```
pwd = input("Enter password:")
correct_pwd = "Hello World"

while pwd != correct_pwd: # as long as it is incorrect
    print("Wrong password! Try again")
    pwd = input("Enter password:")
print("Correct password")
```



Homework Name: Lab1HW1

1. Simple Number Check (2 Points)

Instructions: Write a program that asks the user to enter a number.

- If the number is **less than 5**, print the message: "less than".
- Otherwise (**if it's 5 or more**), print the message: "higher than".

Hint: The `input()` function reads user input as text, so you will need to convert it to a number and compare it.

2. Number Guessing Game (3 Points)

Instructions: Create a simple guessing game. The computer will pick a secret number, and the user will try to guess it.

1. The program should **randomly choose a whole number between 1 and 100** (including 1 and 100).
2. Then, it should ask the user to guess what the number is.
3. The program should print and tell the user if their guess was:
 - "Your guess is **higher** than the secret number."
 - "Your guess is **lower** than the secret number."
 - "You guessed it! The number was", <secret_number>."
4. If the guess is wrong, the program should keep asking the user for a new guess until they get it right.

Hint: You'll need to use the `random` library and the `while` loop for this problem. Please look up and research how to use `random.randint()`.

Homework Instructions

Submission Format:

- Please submit your assignments as either a **.ipynb** (Jupyter Notebook) file or a **.pdf** file.
- If you choose to submit a **.pdf**, make sure all of your code and its output are **clearly visible**.

File Naming Convention:

- You must name your file exactly as shown below:
- YourName_YourStudentID_AssignmentName.FileExtension
- **Example:** Anushka_st126222_Lab1HW.pdf

Late Submissions:

- Please submit all assignments **on time**. Late submissions will be **penalized**.
- Submissions made **within 24 hours after the deadline** will receive **50% penalty**.
- Submissions will **not be accepted** more than **24 hours after the deadline**.

Resource for Further Studies into Python



<https://www.w3schools.com/python/>

Any Questions?

Send Email at: st126222@ait.ac.th